
Encapsulation des données

1 Introduction

En programmation modulaire, on utilise l'encapsulation des traitements : l'implémentation des fonctions publiques est privée. Ces fonctions sont déclarées dans le fichier d'en-tête (partie publique du module) et définies dans le fichier d'implémentation (partie privée du module). Un changement dans le corps de ces fonctions ne nécessite pas de modifier les programmes utilisant le module.

Mais on utilise aussi l'encapsulation des données. L'objectif de ce document est d'illustrer ce principe sur un exemple très simple de module permettant de gérer des points du plan, chaque point étant représenté par un identifiant constitué d'un caractère (le nom du point), son abscisse et son ordonnée.

2 Première version du module sans encapsulation des données

point1.h

```
#ifndef POINT1_H
#define POINT1_H
#include <stddef.h>

struct sPoint
{
    char id;
    double x,y;
};

typedef struct sPoint *tPoint;

extern tPoint newPoint(void);
extern void freePoint(tPoint point);

#endif /* POINT1_H */
```

point1.c

```
#include <stdlib.h>
#include "point1.h"

tPoint newPoint(void)
{
    return malloc(sizeof(struct sPoint));
}

void freePoint(tPoint point)
{
    free(point);
}
```

La structure de données est publique car la définition de la structure `sPoint` a été faite dans le fichier d'en-tête `point1.h`. Un utilisateur du module a donc accès à cette structure de données, c'est-à-dire aux champs de la structure `sPoint`. C'est par exemple le cas du fichier `prog1.c` ci-dessous.

prog1.c

```
#include <stdio.h>
#include "point1.h"

int main(void)
{
    tPoint point;
    point=newPoint();
    point->id='A';
    point->x=3.5;
    point->y=4.2;
    printf("%c_("%.1f,%.1f)\n",
           point->id,
           point->x,
           point->y);
    freePoint(point);
    return 0;
}
```

Les fichiers objets peuvent être produits avec les commandes :

```
gcc -c point1.c
```

```
gcc -c prog1.c
```

L'édition de liens permettant de produire l'exécutable **prog1** peut être effectuée avec la commande :

```
gcc prog1.o point1.o -o prog1
```

L'absence d'encapsulation des données a un inconvénient : toute modification de la structure de données nécessite d'effectuer des modifications des programmes utilisant le module, comme le fichier **prog1.c** dans l'exemple ci-dessus, ainsi qu'une nouvelle production des fichiers objets et de l'exécutable.

3 Deuxième version du module avec encapsulation des données

point2.h

```
#ifndef POINT2_H
#define POINT2_H
#include <stddef.h>

typedef struct sPoint *tPoint;

extern tPoint newPoint(void);
extern void setId(tPoint point,
                 char ident);
extern void setX(tPoint point,
                double xCoord);
extern void setY(tPoint point,
                double yCoord);
extern char getId(tPoint point);
extern double getX(tPoint point);
extern double getY(tPoint point);
extern void freePoint(tPoint point);

#endif /* POINT2_H */
```

point2.c

```
#include <stdlib.h>
#include "point2.h"

struct sPoint
{
    char id;
    double x,y;
};

tPoint newPoint(void)
{
    return malloc(sizeof(struct sPoint));
}

void setId(tPoint point, char ident)
{
    point->id=ident;
}

void setX(tPoint point, double xCoord)
{
    point->x=xCoord;
}

void setY(tPoint point, double yCoord)
{
    point->y=yCoord;
}

char getId(tPoint point)
{
    return point->id;
}

double getX(tPoint point)
{
    return point->x;
}

double getY(tPoint point)
{
    return point->y;
}

void freePoint(tPoint point)
{
    free(point);
}
```

Dans cette version, la structure de données est privée : la définition de la structure `sPoint` est faite dans le fichier d'implémentation `point2.c`.

Les champs de la structure ne sont donc pas accessibles pour les utilisateurs du module. L'accès à ces informations nécessite d'ajouter au module les fonctions publiques de lecture et d'écriture de chacune des informations stockées dans la structure de données¹. Une nouvelle version du programme qui utilise le module `point2` est donnée dans le fichier `prog2.c` ci-dessous.

prog2.c

```
#include <stdio.h>
#include "point2.h"

int main(void)
{
    tPoint point;
    point=newPoint();
    setId(point,'A');
    setX(point,3.5);
    setY(point,4.2);
    printf("%c (%.1f,%.1f)\n",
           getId(point),
           getX(point),
           getY(point));
    freePoint(point);
    return 0;
}
```

La compilation de l'ensemble peut être effectuée grâce aux commandes :

```
gcc -c point2.c
gcc -c prog2.c
gcc prog2.o point2.o -o prog2
```

4 Troisième version du module avec modification de la structure de données

Si on souhaite modifier la structure de données, il suffit de faire les modifications dans le fichier d'implémentation. Le fichier `point3.c` ci-dessous en donne un exemple.

1. En programmation orientée objet, c'est ce que l'on appelle parfois les « *getters* » et les « *setters* », c'est-à-dire les « accesseurs » qui permettent d'accéder aux valeurs des attributs privés.

point3.c

```
#include <stdlib.h>
#include "point2.h"

struct sPoint
{
    char id;
    double coord[2];
};

tPoint newPoint(void)
{
    return malloc(sizeof(struct sPoint));
}

void setId(tPoint point, char ident)
{
    point->id=ident;
}

void setX(tPoint point, double xCoord)
{
    point->coord[0]=xCoord;
}

void setY(tPoint point, double yCoord)
{
    point->coord[1]=yCoord;
}

char getId(tPoint point)
{
    return point->id;
}

double getX(tPoint point)
{
    return point->coord[0];
}

double getY(tPoint point)
{
    return point->coord[1];
}

void freePoint(tPoint point)
{
    free(point);
}
```

Le fichier d'en-tête du module ne doit pas être modifié. Seul le fichier d'implémentation doit l'être : les champs `x` et `y` de la structure `sPoint` ont été remplacés par le tableau `coord`; les corps des fonctions `setX`, `setY`, `getX` et `getY` ont été modifiés en conséquence.

Le programme qui utilise la nouvelle version du module ne doit pas être modifié. Le fichier objet `prog2.o` ne doit pas être mis à jour. Il suffit de mettre à jour le fichier objet du module :

```
gcc -c point3.c
```

et faire une nouvelle édition de liens :

```
gcc prog2.o point3.o -o prog2
```

5 Conclusion

L'encapsulation des données nécessite de rajouter des fonctions permettant d'accéder, en lecture et en écriture, aux informations stockées dans la structure de données. Mais l'implémentation de la structure de donnée peut être modifiée sans conséquence pour les utilisateurs du module.